Programming with Python & Problem Solving (05MC0107)
Lab Manual (2026-27)

Program: M.C.A.

Semester: 1

Name of the Student: Maheta Kunjal Vijaybhai

Enrolment No.: 92600584022

INDEX

Lab	Program	Date	Remarks	Signature
1	Write a program to demonstrate basic input and output operations in Python.	22-7-26		
2	Write a program to illustrate the use of different data types and type casting.	22-7-26		
3	Write a program to perform arithmetic relational and logical operations using Python operators.	22-7-26		
4	Write a program to demonstrate string operations including slicing formatting and built-in string functions.	27-7-26		
5	Write a program to create and manipulate lists using indexing slicing and list comprehensions.	27-7-26		
6	Write a program to illustrate the use of tuples and sets with basic operations.	27-7-26		
7	Write a program to create a dictionary and demonstrate dictionary methods and iteration.	4-8-26		
8	Write a program to explain mutable and immutable objects in Python.	4-8-26		
9	Write a program to define and use userdefined functions with different types of arguments.	4-8-26		
10	Write a program to demonstrate recursion using factorial or Fibonacci series.	11-8-26		
11	Write a program to demonstrate conditional statements using if if-else and if-elif-else.	11-8-26		
12	Write a program to check whether a number is positive negative or zero using nested conditions.	11-8-26		
13	Write a program to generate a multiplication table using a for loop.	18-8-26		
14	Write a program to find the sum of digits of a number using a while loop.	18-8-26		
15	Write a program to demonstrate the use of break continue and pass statements.	18-8-26		

16	Write a program to iterate over lists strings and dictionaries using loops.	18-8-26		
17	Write a program to demonstrate list dictionary and set comprehensions.	20-8-26		
18	Write a program to illustrate variable scope using local global and nonlocal variables.	20-8-26		
19	Write a program to demonstrate iterators and iterables in Python.	20-8-26		
20	Write a program to generate a sequence of numbers using generator functions and yield keyword.	20-8-26		
21	Write a program to create and import a userdefined module.	22-8-26		
22	Write a program to demonstrate different import mechanisms in Python.	22-8-26		
23	Write a program to perform mathematical operations using math module.	22-8-26		
24	Write a program to generate random numbers using random module	23-8-26		
25	Write a program to generate random numbers using random module.	23-8-26		
26	Write a program to perform file and directory operations using os and sys modules.	27-8-26		
27	Write a program to copy move and delete files using shut il module.	30-8-26		
28	Write a program to demonstrate basic regular expression pattern matching.	30-8-26		
29	Write a program to use re module functions such as match search and find all.	5-9-26		
30	Write a program to extract specific information from a text file using regular expressions.	5-9-26		
31				
32				
33				
34				
35				
36				
37				
38				
39				
40				

41				
42				
43				
44				
45				
46				
47				
48				
49				
50				

Practical No. 1
Write a program to demonstrate basic input and output operations in Python.

Aim/ Objective:
To write a Python program that demonstrates **basic input and output operations**, by accepting a number from the user using `input()` and displaying it using `print()`.

Source Code:
```
# Write a program to demonstrate basic input and output operations in Python.

a=input("Enter A number:")
print("The number is :",a)
```

Output:
```
================== RESTART: D:\92600584022\Python\Unit_1\1.py ==================
Enter A number:8
The number is : 8
```

Conclusion:
Thus, the Python program successfully demonstrates **basic input and output operations**. The program takes a number as input from the user and displays the entered number as output.

Practical No. 2
Write a program to illustrate the use of different data types and type casting.

Aim/ Objective:
To understand different data types in Python and demonstrate type casting by converting values from one data type to another.

Source Code:
```
# Write a program to illustrate the use of different data types and type casting.

i=10
flt=10.5
text="Hiii"
bool1=True
print("int :",i,type(i))
print("float :",flt,type(flt))
print("text :",text,type(text))
print("bool :",bool1,type(bool1))
print()
print("Type casting")
a=20
b=float(a)
c=str(b)
d=bool(c
print("Integer to float :",b,type(b))
print("Float to String :",c,type(c))
print("String to Bool :",d,type(d))
```

Output:
```
================== RESTART: D:\92600584022\Python\Unit_1\2.py ==================
int : 10 <class 'int'>
float : 10.5 <class 'float'>
text : Hiii <class 'str'>
bool : True <class 'bool'>

Type casting
Integer to float : 20.0 <class 'float'>
Float to String : 20.0 <class 'str'>
String to Bool : True <class 'bool'>
```

Conclusion :
The program successfully demonstrates Python data types and type casting.

Practical No. 3

Write a program to perform arithmetic relational and logical operations using Python operators.

Aim/ Objective:

To perform and understand arithmetic, relational, and logical operations using Python operators.

Source Code:

```
#  Write a program to perform arithmetic relational and logical operations using Python operator

print("Arithmetic operators")
a=10
b=3
print()
print("Addition :",a+b)
print("Subtraction :",a-b)
print("Multiplication :",a*b)
print("Division :",a/b)
print("Modulus :",a%b)
print("Exponentation :",a**b)
print("Floor division :",a//b)
print()
print("Relational operators")
print()
print("Equal to :",a==b)
print("Not equal to:",a!=b)
print("Less than:",a<b)
print("Greater than:",a>b)
print("Less than equal to:",a<=b)
print("Greater than equal to:",a>=b
print()
print("Logical operators")
print(
print("And :",a>b and b>0)
print("Or :",a>b or b>0)
print("Not :",not(b>0))
```

Output:

```
================== RESTART: D:\92600584022\Python\Unit_1\3.py ==================
Arithmetic operators

Addition : 13
Subtraction : 7
Multiplication : 30
Division : 3.3333333333333335
Modulus : 1
Exponentation : 1000
Floor division : 3

Relational operators

Equal to : False
Not equal to: True
Less than: False
Greater than: True
Less than equal to: False
Greater than equal to: True

Logical operators

And : True
Or : True
Not : False
```

Conclusion :

The program successfully illustrates the use of various Python data types and type conversion.

Practical No. 4
Write a program to demonstrate string operations including slicing formatting and built-in string functions.

Aim/ Objective:
To write a Python program to demonstrate various **string operations** such as concatenation, repetition, indexing, slicing, membership, and comparison.
To demonstrate commonly used **built-in string functions** for manipulating and analyzing strings efficiently.

Source Code:

```
# Write a program to demonstrate string operations including slicing formatting and built-in string functions.

text = "Python"
print("String Operations :- ")
print("Concatenation:" + text + " Programming")
print("Repetion : " + text * 2)
print("Indexing:" + text[0])
print("Slicing :" + text[1:4])

#Membership
print("Py" in text)

# Comparison
print(text == "Python")

# Length
print(len(text))

print("====================================")
print("String built in functions :-")

text = "hello python"

print("Original String:", text)
print("Upper:", text.upper())
print("Lower:", text.lower())
print("Capitalize:", text.capitalize())
print("Title:", text.title())
print("Strip:", text.strip())
print("Replace:", text.replace("python", "World"))
print("Find 'python':", text.find("python"))
print("Count 'o':", text.count("o"))
words = text.strip().split()
print("Split:", words)
print("Join:", "-".join(words))
```

```
print("Startswith ' he':", text.startswith("  he"))
print("Endswith ' ':", text.endswith("  "))
print("Is Alpha:", "Python".isalpha())
print("Is Digit:", text.isdigit())
print("Length:", len(text))
```

Output:

```
================== RESTART: D:\92600584022\Python\Unit_1\4.py ==================
String Operations :-
Concatenation:Python Programming
Repetion : PythonPython
Indexing:P
Slicing :yth
True
True
6
====================================
String built in functions :-
Original String: hello python
Upper: HELLO PYTHON
Lower: hello python
Capitalize: Hello python
Title: Hello Python
Strip: hello python
Replace: hello World
Find 'python': 6
Count 'o': 2
Split: ['hello', 'python']
Join: hello-python
Startswith '  he': False
Endswith '  ': False
Is Alpha: True
Is Digit: False
Length: 12
```

Conclusion:

Thus, the program successfully demonstrates various **string operations and built-in string functions** in Python. It helps in understanding how strings can be accessed, manipulated, searched, and analyzed efficiently.

Practical No. 5

Write a program to create and manipulate lists using indexing slicing and list comprehensions.

Aim/ Objective:

To write a Python program to create and manipulate lists using **indexing, slicing, and list comprehensions**. The program demonstrates how to access, extract, and process list elements efficiently.

Source Code:

```
# Write a program to create and manipulate lists using indexing slicing and list comprehensions.

college = ["MCA","MBA","MCOM","M.ED","M.PHARM"]

#indexing
print("First Field : "+college[0])
print("First Field : "+college[-1])

#slicing
print("Slice fields: ",college[2:])
print("Slice fields: ",college[2:4])
print("Slice fields: ",college[:4])

#List Comprehension
numbers = [1,2,3,4,5]
square = [x*x for x in numbers]

print("Numbers:", numbers)
print("Squares:", square)
```

Output:

```
================== RESTART: D:\92600584022\Python\Unit_1\5.py ==================
First Field : MCA
First Field : M.PHARM
Slice fields:  ['MCOM', 'M.ED', 'M.PHARM']
Slice fields:  ['MCOM', 'M.ED']
Slice fields:  ['MCA', 'MBA', 'MCOM', 'M.ED']
Numbers: [1, 2, 3, 4, 5]
Squares: [1, 4, 9, 16, 25]
```

Conclusion:

Thus, the program successfully demonstrates **list indexing, slicing, and list comprehension** in Python. It helps in understanding how list elements can be accessed and manipulated efficiently.

Practical No. 6
Write a program to illustrate the use of tuples and sets with basic operations.

Aim/ Objective:
To understand and implement **tuples and sets in Python** by performing common operations on them. The program demonstrates accessing tuple elements and modifying sets using built-in operations.

Source Code:
```
#  Write a program to illustrate the use of tuples and sets with basic operations.

t1=(10,20,"hii",2.5,'d')
print("Type :",type(t1))

print("Tuple :",t1)

print("Indexing : ",t1[0])
print("Negative Indexing : ",t1[-1])
print("Slicing : ",t1[2:5])

print("Concatenation : ",t1+("hello",20))
print("Repetition : ",t1*2)
print("Length : ",len(t1))
print("Count : ",t1.count("hii"))

s1={10,20,30,'hiii',2.3}
s1.add(25)
print(s1)
```
Output:
```
================== RESTART: D:\92600584022\Python\Unit_1\6.py ==================
Type : <class 'tuple'>
Tuple : (10, 20, 'hii', 2.5, 'd')
Indexing :  10
Negative Indexing :  d
Slicing :  ('hii', 2.5, 'd')
Concatenation :  (10, 20, 'hii', 2.5, 'd', 'hello', 20)
Repetition :  (10, 20, 'hii', 2.5, 'd', 10, 20, 'hii', 2.5, 'd')
Length :  5
Count :  1
{2.3, 20, 25, 10, 'hiii', 30}
```

Conclusion:
The program successfully demonstrates the use of **tuples and sets** along with their basic operations. It provides an understanding of tuple handling and set manipulation in Python.

Practical No. 7

Write a program to create a dictionary and demonstrate dictionary methods anditeration.

Aim/ Objective:
To create and work with a **dictionary in Python** using different dictionary methods. The program demonstrates accessing keys and values, updating dictionary elements, and iterating through key-value pairs.

Source Code:

```
# Write a program to create a dictionary and demonstrate dictionary methods anditeration.

student = {
    "Name": "Kunjal",
    "Age": 20,
    "Course": "Python"
}

print("Original Dictionary:")
print(student)

print()

print("Keys:")
print(student.keys())

print()

print("Values:")
print(student.values())

print()

print("Items:")
print(student.items())

print()

student.update({"City": "Rajkot"})
print("After Update:")
print(student)

print()
print("Iterating through Dictionary:")
for key, value in student.items():
    print(key, ":", value)
```

Output:

```
================== RESTART: D:\92600584022\Python\Unit_1\7.py ==================
Original Dictionary:
{'Name': 'Kunjal', 'Age': 20, 'Course': 'Python'}

Keys:
dict_keys(['Name', 'Age', 'Course'])

Values:
dict_values(['Kunjal', 20, 'Python'])

Items:
dict_items([('Name', 'Kunjal'), ('Age', 20), ('Course', 'Python')])

After Update:
{'Name': 'Kunjal', 'Age': 20, 'Course': 'Python', 'City': 'Rajkot'}

Iterating through Dictionary:
Name : Kunjal
Age : 20
Course : Python
City : Rajkot
```

Conclusion:

The program successfully demonstrates **dictionary creation, built-in methods, updating elements, and iteration** in Python. It helps in understanding how data stored as key-value pairs can be accessed and processed.

Practical No. 8
Write a program to explain mutable and immutable objects in Python.

Aim/ Objective:
To understand the difference between **mutable and immutable objects in Python** by performing operations on different data types. The program demonstrates which objects can be changed after creation and which require a new object for modification.

Source Code:

```
# Write a program to explain mutable and immutable objects in Python.

print("===== MUTABLE OBJECTS =====")

# List
list1 = [10, 20, 30]
list1.append(40)
print("List:", list1)

# Dictionary
dict1 = {"Name": "Kunjal", "Age": 20}
dict1["City"] = "Rajkot"
print("Dictionary:", dict1)

# Set
set1 = {1, 2, 3}
set1.add(4)
print("Set:", set1)


print("\n===== IMMUTABLE OBJECTS =====")

# Integer
num = 10
num = num + 5
print("Integer:", num)

# Float
f = 12.5
f = f + 2.5
print("Float:", f)

# String
str1 = "Python"
str2 = str1 + " Programming"
print("Original String:", str1)
```

```
print("New String:", str2) #

Tuple
 tuple1 = (10, 20, 30)
 print("Tuple:", tuple1)

 # Boolean
 flag = True
 flag = False
 print("Boolean:", flag)

 # Complex Number
 c = 2 + 3j
 c = c + 1
 print("Complex:", c)
```

Output:

```
================== RESTART: D:\92600584022\Python\Unit_1\8.py ==================
===== MUTABLE OBJECTS =====
List: [10, 20, 30, 40]
Dictionary: {'Name': 'Kunjal', 'Age': 20, 'City': 'Rajkot'}
Set: {1, 2, 3, 4}

===== IMMUTABLE OBJECTS =====
Integer: 15
Float: 15.0
Original String: Python
New String: Python Programming
Tuple: (10, 20, 30)
Boolean: False
Complex: (3+3j)
```

Conclusion:

The program successfully demonstrates **mutable and immutable objects** in Python. It shows that lists, dictionaries, and sets can be modified directly, while integers, floats, strings, tuples, booleans, and complex numbers cannot be changed directly.

Practical No. 9

Write a program to define and use user-defined functions with different types of arguments.

Aim/ Objective:

To create and use **user-defined functions in Python** with different types of arguments. The program demonstrates positional, keyword, default, and variable-length arguments for passing data to functions.

Source Code:

```
# Write a program to define and use user-defined functions with different types of arguments.

#define user-defined functions
def greetings():
    print("Welcome to python...")

# 1.Positional Arguments

def add(a,b):
    print("Sum:",a+b)
add(10,20)

# 2.Keyword Arguments

def student(name,surname):
    print("Your name is ",name,surname)
student(name="kunjal",surname="maheta")

# 3.Default Arguments

def details(name,subject="python"):
    print("welcome to ",subject,name,"!!")
details(name="Kunjal")

# 4.Variable-Length Arguments

def total(*numbers):
    print(numbers)

total("kunjal","maheta")
total(10, 20, 30, 40)
```

Output:

```
================== RESTART: D:\92600584022\Python\Unit_1\9.py ==================
Sum: 30
Your name is  kunjal maheta
welcome to  python Kunjal !!
('kunjal', 'maheta')
(10, 20, 30, 40)
```

Conclusion:

The program successfully demonstrates how **user-defined functions** work with different types of arguments in Python. It helps in understanding flexible ways of passing values to functions and makes programs more organized and reusable.

Practical No. 10
Write a program to demonstrate recursion using factorial or Fibonacci series..

Aim/ Objective:
To write a Python program to demonstrate **recursion** using a factorial function. The program shows how a function can repeatedly call itself until a specified base condition is reached.

Source Code:
```
# Write a program to demonstrate recursion using factorial or Fibonacci series.

def factorial(n): #(3)(2)(1)
    if n == 0 or n == 1: #third time it will becomes true
        return 1
    else:
        return n * factorial(n - 1) #3*f(3-1)  #2*f(2-1)

num = int(input("Enter a number: "))#3
result = factorial(num)
print("Factorial of", num, "=", result)
```

Output:
```
================== RESTART: D:\92600584022\Python\Unit_1\9.py ==================
Sum: 30
Your name is  kunjal maheta
welcome to  python Kunjal !!
('kunjal', 'maheta')
(10, 20, 30, 40)
```

Conclusion:
The program successfully demonstrates the concept of **recursion in Python** by calculating the factorial of a number. It shows how recursive function calls continue until the base condition is satisfied.

Practical No. 11
Write a program to demonstrate conditional statements using if if-else and if-elif-else

Aim/ Objective:
To write a Python program to demonstrate the use of **conditional statements** for evaluating a student's result. The program uses if, if-else, and if-elif-else conditions to determine the appropriate grade based on the percentage.

Source Code:

```
# Write a program to demonstrate conditional statements using if if-else and if-elif-else.

name = input("Enter Student Name: ")

m1 = int(input("Enter marks of Subject 1: "))
m2 = int(input("Enter marks of Subject 2: "))
m3 = int(input("Enter marks of Subject 3: "))
m4 = int(input("Enter marks of Subject 4: "))
m5 = int(input("Enter marks of Subject 5: "))

total = m1 + m2 + m3 + m4 + m5
percentage = total / 5

print("\n----- STUDENT RESULT       ")
print("Student Name :", name)
print("Total Marks  :", total, "/500")
print("Percentage   :", percentage, "%")

if percentage >= 90:
    print("Grade : A+")
elif percentage >= 80:
   print("Grade : A")
elif percentage >= 70:
   print("Grade : B")
elif percentage >= 60:
   print("Grade : C")
elif percentage >= 35:
   print("Grade : D")
else:
    print("Grade : F (Fail)")
```

Output:

```
================== RESTART: D:\92600584022\Python\Unit_2\1.py ==================
Enter Student Name: Dev
Enter marks of Subject 1: 90
Enter marks of Subject 2: 77
Enter marks of Subject 3: 80
Enter marks of Subject 4: 96
Enter marks of Subject 5: 98

----- STUDENT RESULT -----
Student Name : Dev
Total Marks  : 441 /500
Percentage   : 88.2 %
Grade : A
>
```

Conclusion:

The program successfully demonstrates **conditional statements in Python** by calculating the student's total, percentage, and grade. It shows how different conditions can be used to make decisions based on the given marks.

Practical No. 12

Write a program to check whether a number is positive negative or zero using nested conditions.

Aim/ Objective:

To write a Python program to determine whether a given number is **positive, negative, or zero** using conditional statements. The program demonstrates how conditions can be used to classify a number based on its value.

Source Code:

```
# Write a program to check whether a number is positive negative or zero using nested conditions.

a=int(input("Enter number a:"))

if a>0:
   print(" You entered positive number")
elif a<0:
   print("You entered is nagetive number")
else:
   print("You entered is zero")
```

Output:

```
=================== RESTART: D:\92600584022\Python\Unit_2\2.py ===================
Enter number a:9
 You entered positive number
```

Conclusion:

The program successfully checks and identifies whether the entered number is **positive, negative, or zero**. It demonstrates the use of conditional statements for making decisions based on numerical values.

Practical No. 13
Write a program to generate a multiplication table using a for loop

Aim/ Objective:
To write a Python program to generate the **multiplication table of a given number** using a for loop. The program demonstrates repetitive calculations using iteration and supports both integer and decimal values.

Source Code:

```
# Write a program to generate a multiplication table using a for loop.

num = input("Enter a number: ")

if "." in num:

    num = float(num)

else:

    num = int(num)

for i in range(1, 11):

    print(num, "*", i, "=", num * i)
```

Output:

```
================== RESTART: D:\92600584022\Python\Unit_2\3.py ==================
Enter a number: 8
8 * 1 = 8
8 * 2 = 16
8 * 3 = 24
8 * 4 = 32
8 * 5 = 40
8 * 6 = 48
8 * 7 = 56
8 * 8 = 64
8 * 9 = 72
8 * 10 = 80
```

Conclusion:
The program successfully generates the **multiplication table of a given number** using a for loop. It demonstrates the use of iteration to perform repetitive calculations and can handle both integer and decimal numbers.

Practical No. 14
Write a program to find the sum of digits of a number using a while loop

Aim/ Objective:
To develop a Python program that calculates the **sum of individual digits** of a given integer using a while loop. The program also checks whether the entered value is a valid integer.

Source Code:

```
# Write a program to find the sum of digits of a number using a while loop.

num = input("Enter a Number: ")        #1234

if num.isdigit():
    num = int(num)

    sum = 0

    while num > 0:                  #1234>0
        digit = num % 10             #1234%10(remainder)= 4,3,2,1
        sum = sum + digit            #sum=0+4= 4,7,9,10
        num = num // 10              #num=1234//10= 123,12,1

    print("Sum of Digits =", sum)

else:
    print("Warning! Entered number must be an integer.")
```

Output:

```
================== RESTART: D:\92600584022\Python\Unit_2\4.py ==================
Enter a Number: 12345
Sum of Digits = 15
```

Conclusion:
The program successfully finds the **sum of the digits** of the entered number. It demonstrates the use of a while loop along with % and // operators for processing each digit.

Practical No. 15
Write a program to demonstrate the use of break continue and pass statements

Aim/ Objective:
To write a Python program to demonstrate the working of **break, continue, and pass statements**. The program shows how these statements control the execution of a loop in different ways.

Source Code:
```
# Write a program to demonstrate the use of break continue and pass statements.

print("Break Statement")
for i in range(1, 6):
    if i == 3:
        break
    print(i)
print("\nContinue Statement")
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
print("\nPass Statement")
for i in range(1, 6):
    if i == 3:
        pass
    print(i)
```

Output:
```
========== RESTART: D:\9260058402Z\Python\Unit_2\5.py ==========
Break Statement
1
2

Continue Statement
1
2
4
5

Pass Statement
1
2
3
4
5
```

Conclusion:
The program successfully demonstrates the use of **break, continue, and pass** in Python. It shows that break stops the loop, continue skips the current iteration, and pass performs no action.

Practical No. 16
Write a program to iterate over lists strings and dictionaries using loops

Aim/ Objective:
To write a Python program to demonstrate how **loops can be used to iterate through lists, strings, and dictionaries**. The program shows how individual elements, characters, and key-value pairs can be accessed sequentially.

Source Code:

```
# Write a program to iterate over lists strings and dictionaries using loops.

#iterate over lists
fruits=["Apple","Banana","Mango"]

print("Print Fruits:")
for i in fruits:
    print(i)

#iterate over strings

name="python"

for ch in name:
    print(ch)

# Iterate over a Dictionary
print("Iterating over a Dictionary:")
student = {
    "Name": "Kunjal",
    "Age": 20,
    "Course": "MCA"
}

for key,value in student.items():
    print(key, ":", value)
```

Output:

```
================== RESTART: D:\92600584022\Python\Unit_2\6.py ==================
Print Fruits:
Apple
Banana
Mango
p
y
t
h
o
n
Iterating over a Dictionary:
Name : Kunjal
Age : 20
Course : MCA
```

Conclusion:

The program successfully demonstrates the use of **break, continue, and pass** in Python. It shows that break stops the loop, continue skips the current iteration, and pass performs no action.

Practical No. 17
Write a program to demonstrate list dictionary and set comprehensions.

Aim/ Objective:
To write and execute a Python program to demonstrate the use of list, dictionary, and set comprehensions for generating and storing calculated values efficiently from a given collection of numbers.

Source Code:
```
# Write a program to demonstrate list dictionary and set comprehensions.

print("List Comprehension")
numbers = [10, 20, 30, 40, 50]
square_list = [x * x for x in numbers]
print(square_list)
print()
print("Dictionary Comprehension")
square_dict = {x: x * x for x in numbers}
print(square_dict)
print()
print("Set Comprehension")
square_set = {x * x for x in numbers}
print(square_set)
```

Output:

```
================== RESTART: D:\92600584022\Python\Unit_2\17.py ==================
List Comprehension
[100, 400, 900, 1600, 2500]

Dictionary Comprehension
{10: 100, 20: 400, 30: 900, 40: 1600, 50: 2500}

Set Comprehension
{1600, 900, 2500, 100, 400}
>
```

Conclusion:
Thus, the program successfully demonstrates all three types of comprehensions and produces the required list, dictionary, and set of squares.

Practical No. 18

Write a program to illustrate variable scope using local global and nonlocal variables.

Aim/ Objective:

To understand and demonstrate the concept of **local, global, and nonlocal variables** in Python functions and their scope.

Source Code:

```
#Write a program to illustrate variable scope using local global and nonlocal variables.
x = 10
# Global
def outer():
    y = 20
 # Local
   def inner():
       nonlocal y
    y = 30
  # Changes the variable of outer()
    print("Nonlocal variable y =", y)
    print("Global variable x =", x)

inner()
print("Local variable y =", y)

outer()
print("Global variable x =", x)
```

Output:

```
================== RESTART: D:\92600584022\Python\Unit_2\18.py ==================
Nonlocal variable y = 30
Global variable x = 10
Local variable y = 30
Global variable x = 10
```

Conclusion:

The program successfully shows how local variables work within a function, how global variables can be accessed outside functions, and how nonlocal variables can be modified in a nested function.

Practical No. 19

Write a program to demonstrate iterators and iterables in Python

Aim/ Objective:

To illustrate the difference between **iterable objects and iterators** in Python and understand how elements can be accessed sequentially using the `iter()` and `next()` functions.

Source Code:

```
#Write a program to demonstrate iterators and iterables in Python.

#Iterable
numbers = [10,20,30]
print("Iterable object:",numbers)
#iterator
it = iter(numbers)
print("\nElements using iterator:")
print(next(it))
print(next(it))
print(next(it))
```

Output:

```
================== RESTART: D:\92600584022\Python\Unit_2\19.py ==================
Iterable object: [10, 20, 30]

Elements using iterator:
10
20
30
```

Conclusion:

The program successfully demonstrates that a list is an iterable, while `iter()` converts it into an iterator. The `next()` function is then used to retrieve each element one at a time.

Practical No. 20

Write a program to generate a sequence of numbers using generator functions and yield keyword.

Aim/ Objective:

To demonstrate the use of generator functions and the `yield` keyword in Python for producing a sequence of values efficiently.

Source Code:

```
#Write a program to generate a sequence of numbers using generator functions and yield keyword.
def generate_numbers(n):
    for i in range(1, n + 1):
        yield i
n = int(input("Enter the limit: "))
numbers = generate_numbers(n)
print("Generated sequence:")
for num in numbers:
    print(num)
```

Output:

```
================== RESTART: D:\92600584022\Python\Unit_2\20.py ==================
Enter the limit: 4
Generated sequence:
1
2
3
4
```

Conclusion:

The program successfully generates numbers one at a time using a generator. The `yield` keyword allows values to be produced sequentially without storing the entire sequence in memory.

Practical No. 21
Write a program to create and import a userdefined module

Aim/ Objective:
To write and execute a Python program to demonstrate the creation and use of a user-defined module by defining functions in one Python file and importing and using those functions in another Python file.

Source Code:
#Write a program to create and import a userdefined module.

#module_1
```
def add(a, b): return a + b
def multiply(a, b): return a * b
```

#main file
```
import module_1 as m
print("Module imported")
```

Output:
```
>>
========== RESTART: D:\92600584022\Python\Unit_3\21.py ==========
Module imported
>>
```

Conclusion:
Thus, the Python program successfully demonstrates how to create a user-defined module and import it using an alias. The functions defined in the module can be accessed and used in the main program, promoting code reusability and modular programming.

Practical No. 22

Write a program to demonstrate different import mechanisms in Python.

Aim/ Objective:

To write and execute a Python program to demonstrate different import mechanisms, including importing a complete module, importing specific functions, importing multiple functions, and importing a module using an alias.

Source Code:

```
# Write a program to demonstrate different import mechanisms in Python.

import module_1
print("- Using import module:")
print("Addition:", module_1.add(10, 5))

from module_1 import add
print("\n- import function:")
print("Addition:", add(20, 10))

from module_1 import add, multiply
print("\n- import multiple functions:")
print("Addition:", add(30, 10))
print("Multiplication:", multiply(30, 10))

import module_1 as m
print("\n- as alias:")
print("Addition:", m.add(40, 10))
print("Multiplication:", m.multiply(40, 10))
```

Output:

```
================== RESTART: D:\92600584022\Python\Unit_3\22.py ==================
- Using import module:
Addition: 15

- import function:
Addition: 30

- import multiple functions:
Addition: 40
Multiplication: 300

- as alias:
Addition: 50
Multiplication: 400
>>>
```

Conclusion:

Thus, the program successfully demonstrates different import mechanisms in Python. The required functions from the user-defined module are imported and executed successfully using various methods, showing how modules and functions can be accessed efficiently in a Python program.

Practical No. 23

Write a program to perform mathematical operations using math module

Aim/ Objective:

To write and execute a Python program using the `math` module to perform various mathematical operations such as square root, power, absolute value, ceiling, floor, factorial, trigonometric functions, and logarithm on a given number.

Source Code:

```
# Write a program to perform mathematical operations using math module.

import math
num = float(input("Enter a number: "))

print("Square root:", math.sqrt(num))
print("Power (square):", math.pow(num, 2))
print("Absolute value:", math.fabs(num))
print("Ceiling value:", math.ceil(num))
print("Floor value:", math.floor(num))
print("Factorial:", math.factorial(int(num)))
print("Sine:", math.sin(num))
print("Cosine:", math.cos(num))
print("Tangent:", math.tan(num))
print("Logarithm:", math.log(num))
```

Output:

```
================== RESTART: D:\92600584022\Python\Unit_3\23.py ==================
Enter a number: 12
Square root: 3.4641016151377544
Power (square): 144.0
Absolute value: 12.0
Ceiling value: 12
Floor value: 12
Factorial: 479001600
Sine: -0.5365729180004349
Cosine: 0.8438539587324921
Tangent: -0.6358599286615808
Logarithm: 2.4849066497880004
```

Conclusion:

Thus, the program successfully demonstrates the use of Python's built-in `math` module to perform various mathematical operations. Different mathematical functions are applied to the user-provided number, and their results are displayed successfully.

Practical No. 24

Write a program to generate random numbers using random module.

Aim/ Objective:

To write and execute a Python program using the `random` module to generate random integers, random floating-point numbers, random numbers within a specified range, and multiple random integers.

Source Code:

```
#Write a program to generate random numbers using random module.

import random

#Generate random numbers
print("Random integer:", random.randint(1, 100))
print("Random float:", random.random())
print("Random number between 10 and 50:", random.uniform(10, 50))

#Generate multiple random numbers
print("Five random integers:")
for i in range(5):
    print(random.randint(1, 100))
```

Output:

```
================== RESTART: D:\92600584022\Python\Unit_3\24.py ==================
Random integer: 79
Random float: 0.0514326798457827
Random number between 10 and 50: 14.916251877999436
Five random integers:
29
7
54
28
20
```

Conclusion:

Thus, the program successfully demonstrates the use of Python's `random` module to generate different types of random numbers. The program generates random integers, floating-point values, and multiple random numbers within the specified ranges.

Practical No. 25
Write a program to display current date and time using date time module

Aim/ Objective:
To write and execute a Python program using the `datetime` module to display the current date and time of the system.

Source Code:

```
#Write a program to display current date and time using date time module.

from datetime import datetime

#Display current date and time
current_datetime = datetime.now()

print("Current Date and Time:", current_datetime)
```

Output:

```
================== RESTART: D:\92600584022\Python\Unit_3\25.py ==================
Current Date and Time: 2026-09-22 13:03:12.622771
```

Conclusion:
Thus, the program successfully demonstrates the use of Python's `datetime` module to obtain and display the current date and time using the `datetime.now()` function.

Practical No. 26
Write a program to display current date and time using date time module

Aim/ Objective:

To write and execute a Python program using the `os` and `sys` modules to perform basic file and directory operations, such as displaying the current working directory, creating and listing directories, changing directories, and displaying Python version and command-line arguments**.**

Source Code:

```
#Write a program to perform file and directory operations using os and sys modules.

import os
import sys

#Display current working directory
print("Current Directory:", os.getcwd())

#Create a new directory
directory = "MyFolder"
if not os.path.exists(directory):
    os.mkdir(directory)
    print("Directory created:", directory)
else:
    print("Directory already exists:", directory)

#List files and directories
print("Contents of current directory:")
print(os.listdir())

#Display Python version using sys module
print("Python Version:", sys.version)

#Display command-line arguments
print("Command-line Arguments:", sys.argv)

#Change directory
os.chdir(directory)
print("Changed Directory:", os.getcwd())

#Return to parent directory
os.chdir("..")
print("Back to Directory:", os.getcwd())
```

Output:

```
================== RESTART: D:\92600584022\Python\Unit_3\26.py ==================
Current Directory: D:\92600584022\Python\Unit_3
Directory already exists: MyFolder
Contents of current directory:
['21.py', '22.py', '23.py', '24.py', '25.py', '26.py', '27.py', '28.py', '29.py'
, '30.py', 'data.txt', 'module_1.py', 'MyFolder', 'source.txt', 'ss', '__pycache
__']
Python Version: 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942
 64 bit (AMD64)]
Command-line Arguments: ['D:\\92600584022\\Python\\Unit_3\\26.py']
Changed Directory: D:\92600584022\Python\Unit_3\MyFolder
Back to Directory: D:\92600584022\Python\Unit_3
```

Conclusion:

Thus, the program successfully demonstrates the use of Python's `os` and `sys` modules for performing basic file and directory operations and accessing system-related information. The program creates and navigates directories, lists directory contents, and displays Python version and command-line arguments.

Practical No. 27

Write a program to copy move and delete files using shut il module.

Aim/ Objective:

To write and execute a Python program using the `shutil` and `os` modules to copy, move, and delete files.

Source Code:

```
#Write a program to copy move and delete files using shut il module.

import shutil
import os

# Copy a file
shutil.copy("source.txt", "copy.txt")
print("File copied successfully.")

# Move a file
shutil.move("copy.txt", "moved.txt")
print("File moved successfully.")

# Delete a file
os.remove("moved.txt")
print("File deleted successfully.")
```

Output:

```
================== RESTART: D:\92600584022\Python\Unit_3\27.py ==================
File copied successfully.
File moved successfully.
File deleted successfully.
>
```

Conclusion:

Thus, the program successfully demonstrates the use of Python's `shutil` module to copy and move files using `shutil.copy()` and `shutil.move()`, and the `os` module to delete a file using `os.remove()`.

Practical No. 28
Write a program to demonstrate basic regular expression pattern matching

Aim/ Objective:
To write and execute a Python program using the `re` module to demonstrate basic regular expression pattern matching and search for a 10-digit phone number in a given text.

Source Code:

```
#Write a program to demonstrate basic regular expression pattern matching.

import re

# Sample text
text = "My phone number is 9876543210."

# Search for a 10-digit phone number
pattern = r"\d{10}"

match = re.search(pattern, text)

if match:
    print("Pattern matched!")
    print("Phone number:", match.group())
else:
    print("Pattern not matched.")
```

Output:

```
================== RESTART: D:\92600584022\Python\Unit_3\28.py ==================
Pattern matched!
Phone number: 9876543210
```

Conclusion:
Thus, the program successfully demonstrates the use of Python's `re` module to search and match a 10-digit phone number using the regular expression pattern `\d{10}` and display the matched number.

Practical No. 29

Write a program to use re module functions such as match search and find all

Aim/ Objective:

To write and execute a Python program using the `re` module functions such as `match()`, `search()`, and `findall()` to perform regular expression pattern matching in a given text.

Source Code:

```
#Write a program to use re module functions such as match search and find all.

import re

text = "Python is easy to learn. Python is powerful and popular."

result = re.match("Python", text)
if result:
    print("match():", result.group())
else:
    print("match(): No match found")


result = re.search("powerful", text)
if result:
    print("search():", result.group())
else:
    print("search(): No match found")

result = re.findall("Python", text)
print("findall():", result)
```

Output:

```
================== RESTART: D:\92600584022\Python\Unit_3\29.py ================
match(): Python
search(): powerful
findall(): ['Python', 'Python']
```

Conclusion:

Thus, the program successfully demonstrates the use of the `re` module functions `match()`, `search()`, and `findall()` to find and display specific patterns in the given text.

Practical No. 30

Write a program to extract specific information from a text file using regular expressions

Aim/ Objective:

To write and execute a Python program using the `re` module to extract specific information such as email addresses, phone numbers, and dates from a text file using regular expressions.

Source Code:

```
#Write a program to extract specific information from a text file using regular expressions.

import re
with open("data.txt", "r") as file:
    text = file.read()

emails = re.findall(r'[\w.-]+@[\w.-]+\.\w+', text)
phone_numbers = re.findall(r'\b\d{10}\b', text)
 dates = re.findall(r'\b\d{2}-\d{2}-\d{4}\b', text)

print("Email Addresses:")
for email in emails:
    print(email)

print("\nPhone Numbers:")
for phone in phone_numbers:
    print(phone)

print("\nDates:")
for date in dates:
    print(date)
```

Output:

```
========== RESTART: D:\92600584022\Python\Unit_3\30.py ==========
Email Addresses:
rahul@gmail.com
amit@yahoo.com

Phone Numbers:
9876543210
8765432109

Dates:
15-08-2005
20-01-2004
```

Conclusion:

Thus, the program successfully demonstrates the use of Python's `re` module to read a text file and extract email addresses, phone numbers, and dates using suitable regular expression patterns.